Code organization

Scope

In every programming language, we use names to refer to things – functions, variables, objects, and so on. This link between a name and a thing is called **name binding**.

Scope

In every programming language, we use names to refer to things – functions, variables, objects, and so on. This link between a name and a thing is called **name binding**.

Name bindings are context specific. The variable `foo` can refer to one thing in one program and to another thing in another program.

Scope

In every programming language, we use names to refer to things – functions, variables, objects, and so on. This link between a name and a thing is called **name binding**.

Name bindings are context specific. The variable `foo` can refer to one thing in one program and to another thing in another program.

Scope rules are the rules that determine where a specific name binding is valid, and constitute some of the biggest differences between programming languages.

Basic scoping

```
i = 4
def doublei():
    return i * 2
print(doublei())
# 8
```

Basic scoping (lambda function)

```
i = 4
doublei = lambda : i * 2
print(doublei())
# 8
```

Scripts

In Python, scripts can be run directly from the command line. The **workspace** is usually one scope. Scripts work in the workspace scope.

Scripts

In Python, scripts can be run directly from the command line. The **workspace** is usually one scope. Scripts work in the workspace scope.

This has mostly disadvantages. Most importantly, running a script twice can easily give different results if a variable was changed in the course of the script.

Scripts

In Python, scripts can be run directly from the command line. The **workspace** is usually one scope. Scripts work in the workspace scope.

This has mostly disadvantages. Most importantly, running a script twice can easily give different results if a variable was changed in the course of the script.

The main use case of a script is if you are automating an entire session (e.g., running a file from the command line or running in batch mode on the cluster). In general, try to avoid calling one script from another.

Regular functions

A `def` introduces a **function scope**. Names you bind inside the function (parameters and variables you assign there) are **local** to that call. They do not overwrite names in the enclosing script or module unless you go out of your way to do so.

Regular functions

A `def` introduces a **function scope**. Names you bind inside the function (parameters and variables you assign there) are **local** to that call. They do not overwrite names in the enclosing script or module unless you go out of your way to do so.

```
limit = 100

def clip(x):
    # `x` and `y` are local; `limit` is read from the enclosing scope
    y = max(0, min(x, limit))
    return y
```

Same name, two bindings (shadowing)

If you **assign** to a name inside a function, Python treats that name as **local** for the whole function body. That can **hide** an outer name with the same spelling – a common source of confusion.

Same name, two bindings (shadowing)

If you **assign** to a name inside a function, Python treats that name as **local** for the whole function body. That can **hide** an outer name with the same spelling – a common source of confusion.

```
count = 0

def bump():
    count = count + 1  # Error: local `count` is assigned, so it is not re
```

Same name, two bindings (shadowing)

If you **assign** to a name inside a function, Python treats that name as **local** for the whole function body. That can **hide** an outer name with the same spelling – a common source of confusion.

```
count = 0

def bump():
    count = count + 1  # Error: local `count` is assigned, so it is not re
```

Use different names, pass values explicitly.

Module vs function scope

At **module level** (top of the file, column 0), assignments create **global** names for that file. **Inside** a function, new assignments are local; **reads** walk outward (function $\rightarrow$ enclosing functions $\rightarrow$ module $\rightarrow$ builtins) until a binding is found.

Module vs function scope

At **module level** (top of the file, column 0), assignments create **global** names for that file. **Inside** a function, new assignments are local; **reads** walk outward (function → enclosing functions → module → builtins) until a binding is found.

```
API_URL = "https://example.com/api"

def fetch(path):
    # reads API_URL from module scope; `path` is local
    return API_URL + path
```

Objects, names, and arguments

Scope governs which **names** exist where. **Objects** (numbers, strings, lists, class instances, . . .) live in memory; a name is a **reference** to an object. Assignment `name = value` binds the name to whatever object `value` refers to.

Objects, names, and arguments

Scope governs which **names** exist where. **Objects** (numbers, strings, lists, class instances, . . .) live in memory; a name is a **reference** to an object. Assignment `name = value` binds the name to whatever object `value` refers to.

When you **call** a function, each parameter becomes a **new local name** in the function scope. Python passes **references to objects** (sometimes described as pass-by-assignment or pass-by-object-reference): the caller's argument and the callee's parameter are two names that can refer to the **same** object for the duration of the call.

Mutating

If the object is **mutable**, a function can **change its contents** through the parameter name, and the caller will see those changes – both names still point at the same object.

Mutating

If the object is **mutable**, a function can **change its contents** through the parameter name, and the caller will see those changes – both names still point at the same object.

```
def append_one(items):
    items.append(1)  # mutates the list object

xs = []
append_one(xs)
# xs is now [1]
```

Rebinding

If you **assign** to the parameter name (`items = ...`), you only **rebind** the local name; you do not replace the caller's variable.

Rebinding

If you **assign** to the parameter name (`items = ...`), you only **rebind** the local name; you do not replace the caller's variable.

```
def reset(items):
    items = []  # local name now refers to a new list; caller unchanged

xs = [1, 2]
reset(xs)
# xs is still [1, 2]
```

Rebinding

If you **assign** to the parameter name (`items = ...`), you only **rebind** the local name; you do not replace the caller's variable.

```
def reset(items):
    items = []  # local name now refers to a new list; caller unchanged

xs = [1, 2]
reset(xs)
# xs is still [1, 2]
```

Immutable values (e.g. `int`, `str`, `tuple`) cannot be changed in place; "updates" always create new objects and only affect what the **local** name refers to unless you return the value or mutate a shared mutable container.

Anonymous functions

You can also define functions "on the fly" inside a script or another function. Such **anonymous functions** can see the variables in their parent scope but can't change them. Anonymous functions can be saved as variables, which will be incredibly useful soon.

Anonymous functions

You can also define functions "on the fly" inside a script or another function. Such **anonymous functions** can see the variables in their parent scope but can't change them. Anonymous functions can be saved as variables, which will be incredibly useful soon.

```
>>> myAnonFun = lambda x, y: x+y
```

Anonymous functions can update variables

In some languages, anonymous functions store scope when they are created. Not Python:

Anonymous functions can update variables

In some languages, anonymous functions store scope when they are created. Not Python:

```
>>> i = 4
>>> timesi = lambda k : i * k
>>> print(timesi(3))  # 12
>>> i = 3
>>> print(timesi(3))  # 9
```

Anonymous functions can update variables

In some languages, anonymous functions store scope when they are created. Not Python:

```
>>> i = 4
>>> timesi = lambda k : i * k
>>> print(timesi(3))  # 12
>>> i = 3
>>> print(timesi(3))  # 9
```

Compare MATLAB:

Anonymous functions can update variables

In some languages, anonymous functions store scope when they are created. Not Python:

```
>>> i = 4
>>> timesi = lambda k : i * k
>>> print(timesi(3))  # 12
>>> i = 3
>>> print(timesi(3))  # 9
```

Compare MATLAB:

```
>>> i = 4; timesi = @(k) i * k;
>>> timesi(3)  % 12
>>> i = 3; timesi(3))  % 12 (!!)
```

Some things you should not do but should know about

Forced scope

Warning *This is only for your information, you should* ***almost never*** *force scope in your own work.*

Forced scope

> ***Warning*** *This is only for your information, you should* ***almost never*** *force scope in your own work.*

In most languages, you can force variables into another scope. For example, you can declare `global` variables, or you can store variables somewhere where all functions can access them (e.g., in a file, in the operating system, or in the interface object). **This is generally a bad idea and you should never break scope.**

Default inputs

Warning *This is only for your information, you should* ***almost never*** *use default input values in your own work.*

Default inputs

> ***Warning*** *This is only for your information, you should* ***almost never*** *use default input values in your own work.*

Python functions can assign default values in their definition.

Default inputs

> ***Warning*** *This is only for your information, you should* ***almost never*** *use default input values in your own work.*

Python functions can assign default values in their definition.

```
def thisIsFun(myIn = 1, secondIn = "-"):
    # ... stuff
```